dns-tools Documentation

Release 5.4.1

Gene C

Oct 26, 2026

DNS-TOOLS MANUAL

1 dns_tools **1**
- 1.1 Overview . . . 1
- 1.2 Types of DNSSEC keys . . . 2
- 1.3 Key Rolling . . . 2

2 The Tools **3**
- 2.1 Testing Mode . . . 3
- 2.2 Creating Keys . . . 5
- 2.3 Updating dns zone files . . . 6
- 2.4 Options . . . 6
- 2.5 Update your DNS to use signed zone file . . . 8

3 FAQ **9**
- 3.1 Why is name not dnssec_tools? . . . 9

4 Appendix **11**
- 4.1 Dependencies . . . 11
- 4.2 Build Manually . . . 11
- 4.3 License . . . 11

CHAPTER

ONE

DNS_TOOLS

1.1 Overview

Provides DNS management tools including automating key updates (rolls) for DNSSEC.

dns-tools simplifies DNSSEC deployment very straightforward. It makes it simple to update and re-sign any changes to DNS zone files. Since keys should be updated periodically, this is done with simple cron file to update the keys and publish old and new keys. Then after sufficient time has passed for DNS servers to udpate and have the new keys, the old ones are removed. Key rolling is fully automated.

DNSSEC can be a little tricky especially rolling the keys.

If you're implementing DNSSEC then you may want to implement DANE SMTP for email once DNSSEC is running. If so, you may be interested in ssl-mgr tool which simplifies key and certificate management including DANE SMTP. It is also available on Arch AUR.

1.1.1 Documentation

The manual provides detailed information and is available in both HTML and PDF formats. Both are installed under */usr/share/dns_tools/docs*.

The manual is also available at: readthedocs.

1.1.2 Signed Source

All git tags are signed by *<arch@sapience.com>* and the public key is available via WKD or download from website (same domain as key):

sapience.com.

After key is installed on the keyring then the PKGBUILD source line ending with *?signed* can be used. Manually verification may also be done with *git tag -v <tag-name>*

1.1.3 Available

- Github
- Archlinux AUR

The Archlinux PKGBUILD is also in the packaging directory.

To build manually, see Appendix *Build Manually*.

======== Background ==========

1.2 Types of DNSSEC keys

A short summary of DNSSEC keys.

- **Zone Signing Key (ZSK)**
  This is used to sign dns zone files. It is advisable to update this periodically, perhaps every 1 to 3 months. The mechanism to update the key requires some care and is known as *rolling* the keys. The tools make this straightforward. More on this later.
- **Key Signing Key (KSK)**
  This signs the zone signing key - and its this key that must be registered with the domain registrar for the root servers. The requirement ensures that there is an appropriate chain of trust from the root dns servers on down. Most, if not all registars now support DNSSEC - Squarespace (previously Google Domains) does for example.
  This requirement means there is a manual step whenever the KSK changes, which is to update the root servers with the new information. KSK should be rolled occasionally, in spite of the manual step, perhaps every 1-3 years, and the corresponding DS (Delegation Signer) record for the new KSK should be uploaded to the domain registrar.
  See the *Creating Keys* section about generating the DS records you will need.

1.3 Key Rolling

The typical approach to doing this is accomplished in 2 basic steps.

- **Phase 1**
  Create a new key, called *next*, then sign the zone files using this key as well as the current key (we call this *curr* for short). With the records now double signed the existing key remains valid.
- **Phase 2**
  After a period sufficiently longer than the TTL for the zone, say 2 x TTL, then rename the *next* key to be the *curr* key. Resign using just this key. This gives time for DNS servers to catch up with the new key before the old one is removed.

Rolling KSK and ZSK is basically the same, but for KSK, the DS records must be uploaded to the domain registrar. In Phase 1 both old and new DS should be uploaded and in Phase 2 just the new (now current) KSK DS. The tool creates the DS records but uploading them to registrar must be done manually.

CHAPTER

TWO

THE TOOLS

The following tools are provided.

- **dns-tool**
  This handles all DNSSEC related operations including key creation and rolling, and using those keys to sign the dns zone files.
- **dns-prod-push**
  This tool make it simple to push signed and/or unsigned dns zone files from the signing server to the production area for each primary dns server. the DNS primary server(s) should be on same machine or reachable via ssh. It also restarts those servers when appropriate.
- **dns-serial-bump**
  A standalone executable to check the validity and bump the serial number in the SOA of a dns zone file.

2.1 Testing Mode

Each tool supports a test mode engaged with the *-t, –test* option. When run in test mode, actions are printed instead of actually being done.

When running in test mode nothing is done, which can lead to things seemingly being strange. For example, when testing rolling or generation of *next* keys, the code later checks for any missing keys. Now in test mode they can be missing since they were not actually created when they would normally be. So now you can see messages about keys being generated a second time. They wont be in non-test mode of course as nothing would be missing.

For testing, I find it convenient to change the production dns zone directores to something like */tmp/dns* - and then run the tests without *-t*. This does everything as asked, but instead of pushing to the real dns servers, the files are pushed to the test production directory. When doing this, you can drop the *–dns_restart* option so as to skip restarting the dns servers - which is not needed obviously.

2.1.1 Example Usage

N.B. :

- **Must run on signing server**
  The tools must be run on the signing server which is defined in the config file. To minimize chance of an accident, the code will refuse to run if that is not the case.

- **Run as root**
  - operations require effective root user:
  - Changing the ownership permisions of staging zones to *dns_user* and *dns_group*.
  - Preserving ownership when files rsync –owner to dns server(s)
- **Zone serial numbers should be in canonical format for serial bump to work properly**
  i.e. yyymmddnn where yyymmdd is date and nn is a 2 digit counter from 00 to 99 If not code will do best it can to migrate to canonical format if possible. It will warn of non-standard or invalid serials and replace them with valid serials. A valid serial is all numbers and must be expressable as 32 bits. You can use the *dns-serial-bump –check zonefile* to check for valid serial.

dns-tool supports 2 primary servers - an internal DNS server and an external server. The internal server may also serve additional unsigned zones, typically RFC1918 and their reverse zones. There can be unsigned zones for external server too of course and if there are, they will be pushed along with all the other signed zones.

The external primary is how the outside world views DNS for each domain. As usual once a primary dns server is updated, it's secondaries will get updated automatically via IXFR/AXFR.

The tool is driven by a straightforward config file which is first looked for in current directory under *./conf.d/config* and if not available there it should be in */etc/dns_tools/conf.d/config*.

The config file holds the information about where all the relevant files are kept and the command to use to restart the dns servers, the DNS server hosts and so on.

Copy the sample config file and edit it for your needs:

```
cd /etc/dns_tools
cp conf.d/config.sample conf.d/config
```

Edit the config file to suit your needs. Set the *work_dir* to wherever you want to keep the internal/external zone files and the keys. The sample config uses */etc/dns_tools* for the working directory. Relative directory names are always relative to the working directory.

The *work_dir* holds all the data and is the source for all key and zone information. Signed and unsigned zone files are pushed from the working dir to each of the DNS servers. Internal and external dns zone files are kept in their own directories. e.g.:

```
<work_dir>/internal/staging/zones
```

The *ldns* package has standalone tools which handle key generation and signing the zone files.

With that background information, and under the assumption that the domain registrar already has the ksk required information then to roll ZSK using dns_tools would be simply:

```
/usr/bin/dns-tool --zsk_roll_1
/usr/bin/dns-prod-push --dns_restart --to_production
```

and after couple hours or similar, the second phase would be accomplished using:

```
/usr/bin/dns-tool --zsk_roll_2
/usr/bin/dns-prod-push --dns_restart --to_production
```

Of course in practice each of these would be run from cron - I run them monthly. A sample cron file is provided in */etc/dns_tools/cron/dnssec-roll.cron*. And for convenience, it uses the above commands wrapped in shell scripts:

```
/etc/dns_tools/scripts/zsk-roll-1.sh
/etc/dns_tools/scripts/zsk-roll-2.sh
```

2.2 Creating Keys

To get things started simply create the KSK and ZSK keys and then upload the DS key info to the domain registrar. To generate a new set of keys simply run:

```
/usr/bin/dns-tool --gen_ksk_curr --gen_zsk_curr
```

All the keys will be under the *keys* directory. For each domain, the info needed for the domain registrar will be found in the file:

```
<work_dir>/keys/<domain>/ksk/curr.all.ds
```

By default all the domains in the config are processed. To process a one or more specific domains just put them on the command line. Domains listed on command line will override the config file.

All zone files for both internal and external dns should be available as specified in the config file. See the sample config for more details.

2.2.1 KSK Keys and DS to root servers

When you create KSK keys a set of DS keys will be generated automatically. These actually come in different hash types:

- **1 : SHA-1** - Mandatory RFC3658 : weak, dont use
- **2 : SHA-256** - Mandatory RFC4509 : the default and saved in curr.ds
- **3 : gost R 34.11-94** - Deprecated RFC5933 :
- **4 : SHA-384** - Optional RFC6605 : slower but more secure hash
- **5 : GOST R 34.11-2012** - OPTIONAL RFC9558 : Russian equivalent to SHA-256
- **6 : SM3** - Optional RFC9563 : Chinese equivalent to SHA-256.
- **7 to 255** - Unassigned

We generate the types *1*, *2* and *4*. We default to SHA-256

These are saved into the *<work_dir>/keys/<domain>/ksk/* directory.

In addition to *curr.ds*, *curr.all.ds* contains **sha1**, **sha256** and **sha384**. Choose one or more of these to upload to your domain registrar.

Its good to get this uploaded and available from the root servers soon as your KSK keys are ready and before you push any signed zones out. This is the only manual step. And if/when you roll your ksk, then it needs to be repeated with the new DS key info.

I recommend uploading both sha256 (type 2) and sha512 (type 4) keys. Associated with each the *curr.all.ds* file will also have a numerical Id, which you'll need to share with your registrar. Note that it

can take some time for the root servers to get updated with your new KSK - which is fine. Just means that your DNS will be non-dnssec until they get the KSK pushed out to the world. Once that happens, then dns clients will see the KSK and dnssec will be operational.

Everthing else should be handled automatically by dns-tool.

2.3 Updating dns zone files

Whenever you update any zone files, they must be resigned. Make any zone file changes in the zone staging directories. i.e.

```
<work_dir>/internal/staging/zones
<work_dir>/external/staging/zones
```

You don't need to bump serial number, the tool will do it for you, though its benign to do so. When you're done with the changes then to resign and push just run:

or use the convenience wrapper script for these 2 commands by running:

```
/etc/dns_tool/resign.sh
```

This also takes optional arguments:

- –serial_bump
- list of domains. If none listed, then uses all domains in config file.

2.4 Options

2.4.1 dns-tool

Handles key generation, zone signing and key rolls.

While there are many options, majority are more for testing or special needs. The most common / frequently used options are :

*test *print_keys *sign zsk_toll_1 *zsk_roll_2*

Full list of options:

```
* positional arguments:
   zones                          Zones/domains this action applies to (over rides the configX
↪file)

   -h, --help                     show this help message and exit
  --theme THEME                   Color theme: dark, light or none)
  --sign                          Sign with curr keys (ksk and zsk)
  --gen-ksk-curr, --gen_ksk_curr  Generate KSK for curr
  --gen-ksk-next, --gen_ksk_next  Generate KSK for next
  --gen-zsk-curr, --gen_zsk_curr  Generate ZSK for curr
  --gen-zsk-next, --gen_zsk_next  Generate ZSK for next
  --keep-include, --keep_include  Keep temp file $INCLUDE expanded
  --ksk-algo, --ksk_algo KSK_ALGO Key algorithm for KSK)
  --ksk-roll-1, --ksk_roll_1 KSK  Phase 1 roll: old & new. ** Plesae Update registrar
  --ksk-roll-2, --ksk_roll_2 KSK  Phase 2 roll - new only
  --print-keys, --print_keys      Print keys (curr and next)
```

(continues on next page)

(continued from previous page)

```
  --serial-bump, --serial_bump      Bump all serials
  --sign-ksk-next, --sign_ksk_next  Sign with both next and curr ksk
  --sign-zsk-next, --sign_zsk_next  Sign with both next and curr zsk
  --zsk-algo, --zsk_algo ZSK_ALGO   Key algorithm for ZSK)
  --zsk-roll-1, --zsk_roll_1 ZSK    Phase 1 roll - old and new
  --zsk-roll-2, --zsk_roll_2 ZSK    Phase 2 roll - new only
  -t, --test                        Test mode - print but dont do
  -v, --verb                        More verbosity
```

Notes:

- Bump all serials is not usually needed since its auotmatic. And doing this implies *–sign* so that signed zones stay consistent.
- Supported KSK key algorithm algos default is ED25519. Also available ECDSAP256SHA256, ECDSAP384SHA384, ED448

2.4.2 dns-prod-push

Push signed and/or unsigned zones to the dns server(s):

```
positional arguments                one or more domains here will override config file.

  -h, --help                        show this help message and exit
  --theme THEME                     Color theme: dark, light or none
  --dns-restart, --dns_restart      Restart the DNS server after zones updated
  --int-ext, --int_ext INT_EXT      What to push: internal, external or both (both)
  --to-production, --to_production  Copy zone files from staging to live production
  -t, --test                        Test mode - print but dont do
  -v, --verb                        More verbosity
```

Notes:

- dns_restart
  Uses the config variable *dns_restart_cmd*. For example for nsd, set this to:

  ```
  dns_restart_cmd = "/usr/bin/systemctl restart nsd"
  ```

2.4.3 dns-serial-bump

Bump the serial number of a DNS zone file:

```
dns-serial-bump [-c] <zonefile [zonefile ..]>

positional arguments:
  zonefile     Zone file(s) with serial numbers to increment

options:
  -h, --help   show this help message and exit
  -c, --check  Check serial from zonefile
```

Notes:

- Check Shows current and the bumped serial number for each zonefile. Does not change any serial numbers, just displays them. Without *check* option each zonefile will be updated with new serial.

2.5 Update your DNS to use signed zone file

When you're ready to switch your dns to dnssec then all that's needed is change the primary server config to point to the signed zone file rather than the unsigned.

For nsd this would be of the form:

```
zone:
    name:       example.com
    #zonefile:  %s                      # unsigned
    zonefile:   %s.signed/zone          # signed
    include-pattern: "tosecondary"      # notify all secondary servers
```

CHAPTER

THREE

FAQ

3.1 Why is name not dnssec_tools?

This is a good question. I did give some thought to this and ended up with the more generic name.

My thinking is this. Since the tool is really about managing DNS zones in one place and not just about keys/signing I went with the more generic name combined with the addition of DNSSEC keyword.

There are three basic parts to the tools:

- Check the validity and increment the serial number in the SOA section of zonefile.
- Push zone files to primary DNS servers (internal and external facing servers) and restart them.
- Generate and manage KSK and ZSK keys and use them to sign zones.

While all of them are needed to provide automation of key rolls, the first two items above are not specific to DNSSEC. That said the bulk of the code deals with the more complex DNSSEC tasks.

CHAPTER

FOUR

APPENDIX

4.1 Dependencies

Run Time :

- python (3.14 or later)
- ldns
- lockmgr
- pyconcurrent

Building Package:

- git
- meson
- meson-python
- rsync

4.2 Build Manually

To build it manually, clone the repo and do:

```
./scripts/do-build
./scripts/do-install <destination-directory>
```

4.3 License

Created by Gene C. and licensed under the terms of the GPL-2.0-or-later license.

- SPDX-License-Identifier: GPL-2.0-or-later
- SPDX-FileCopyrightText: © 2023-present Gene C <arch@sapience.com>